Building Better Pipelines on Databricks

Mohit Batra
DATA ENGINEER

linkedin.com/in/mohitbatra

Overview

Databricks APIs

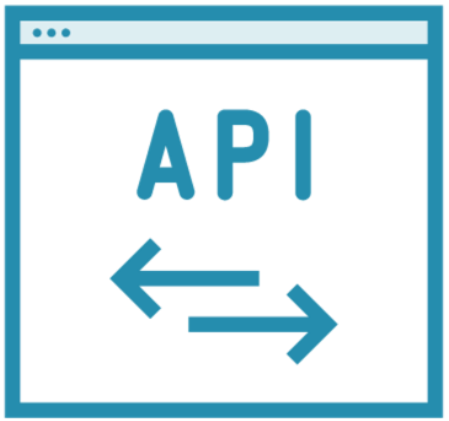
- Services for which it is available
- Invoke APIs using Powershell
- Read data using API call
- Trigger an action using API call

Delta Lake

- Why is it required?
- How to use it?
- How does it work?
- Features
- Use Cases

Using Databricks APIs

Databricks APIs



REST APIs to access Databricks services

Automate CI/CD pipelines

APIs for:

- Workspace
- Clusters/Pools
- Jobs
- DBFS
- Libraries
- Executing commands etc.

```
$info =
@{
   Uri = "https://<location>.azuredatabricks.net/api/<endpoint>"
   Headers = @{"Authorization" = "Bearer <access token>"}
 }
Invoke-RestMethod -Method '<http method>' @info
```

Databricks API Call Using Powershell

location

- Deployment Region (eastus2)

endpoint

- API URL (2.0/jobs/list)

access token

- User Settings → Access token

http method

- GET, POST, PUT, DELETE

Understanding Delta Lake

Data Lake is a central repository to store all types of data at any scale

Data Lake Challenges

Dirty Reads

Multiple processes reading and writing on same files

Data Inconsistency

Job failures may lead to only partial writes in data lake

Slow Performance

With growth, there is increase in number of files, which scatters data

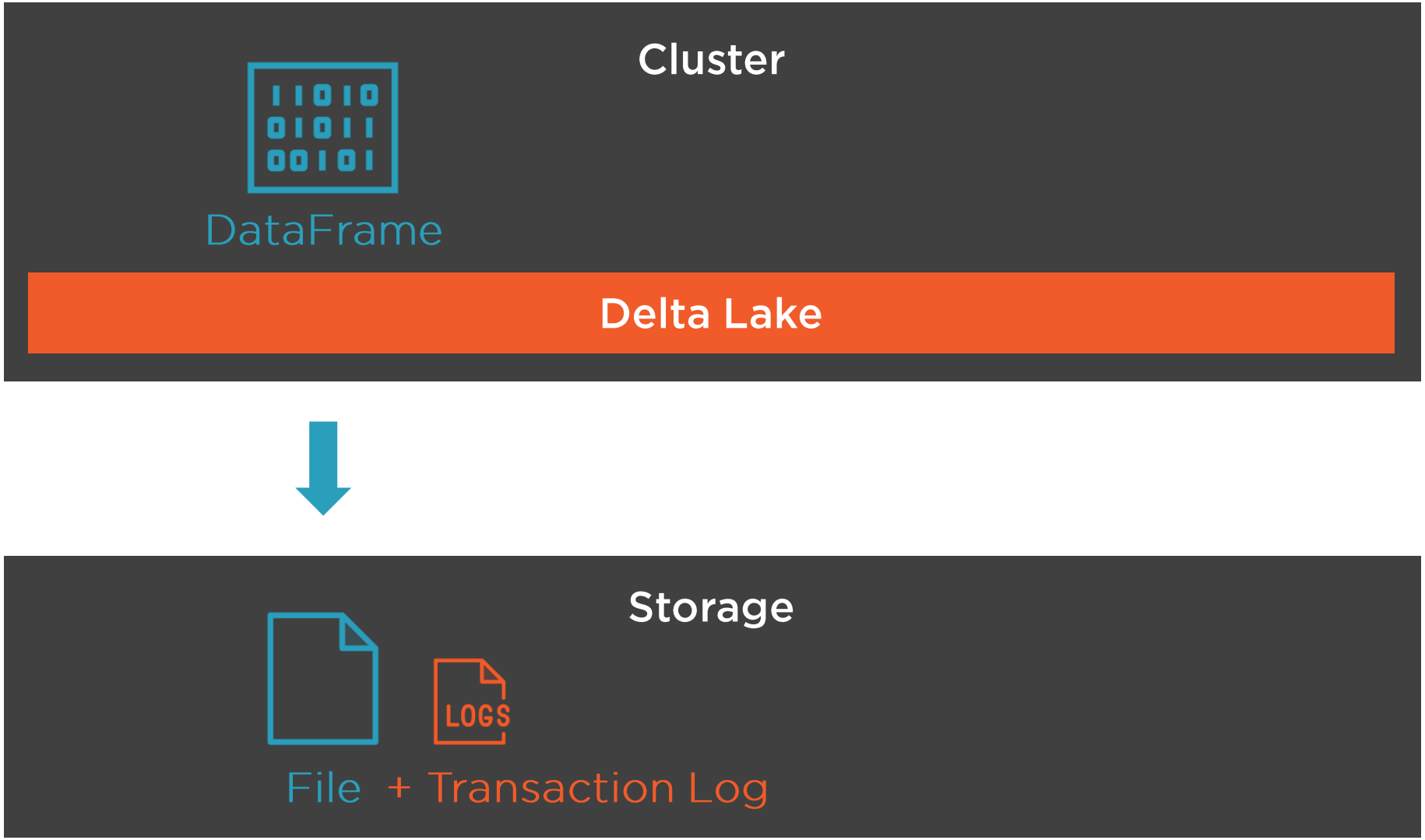
Record Updates

Read entire dataset, update records & save back, even for a single update

Delta Lake is an open source storage layer that brings reliability to data lakes

```
yellowTaxiTripDataDF
      .write
      .format("delta")
      .save("/mnt/datalake/YellowTaxiTripData.parquet")
```

Save DataFrame in Delta Format


Cluster
DataFrame
Delta Lake
Storage
LOGS
File + Transaction Log

Transaction Log

Also called a Delta log

Ordered log of every transaction performed on table since its inception

Used to define final view of the table

Contains commit information:

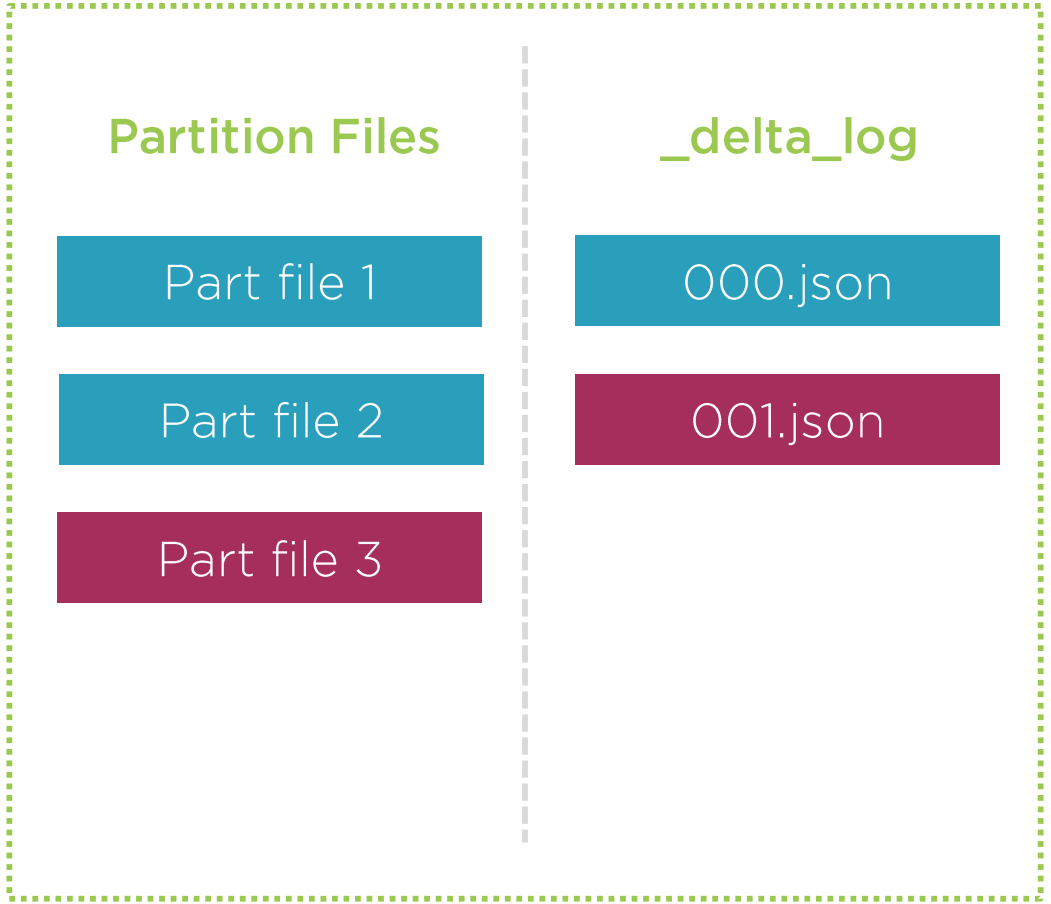
- Operation performed
- Predicates
- Partition files affected (added/removed)

Simultaneous Read / Write

Writer Process

1. Write partition files
2. Update transaction log

Write new changes via Part file 3



Reader Process

1. Read transaction log
2. Read partition files

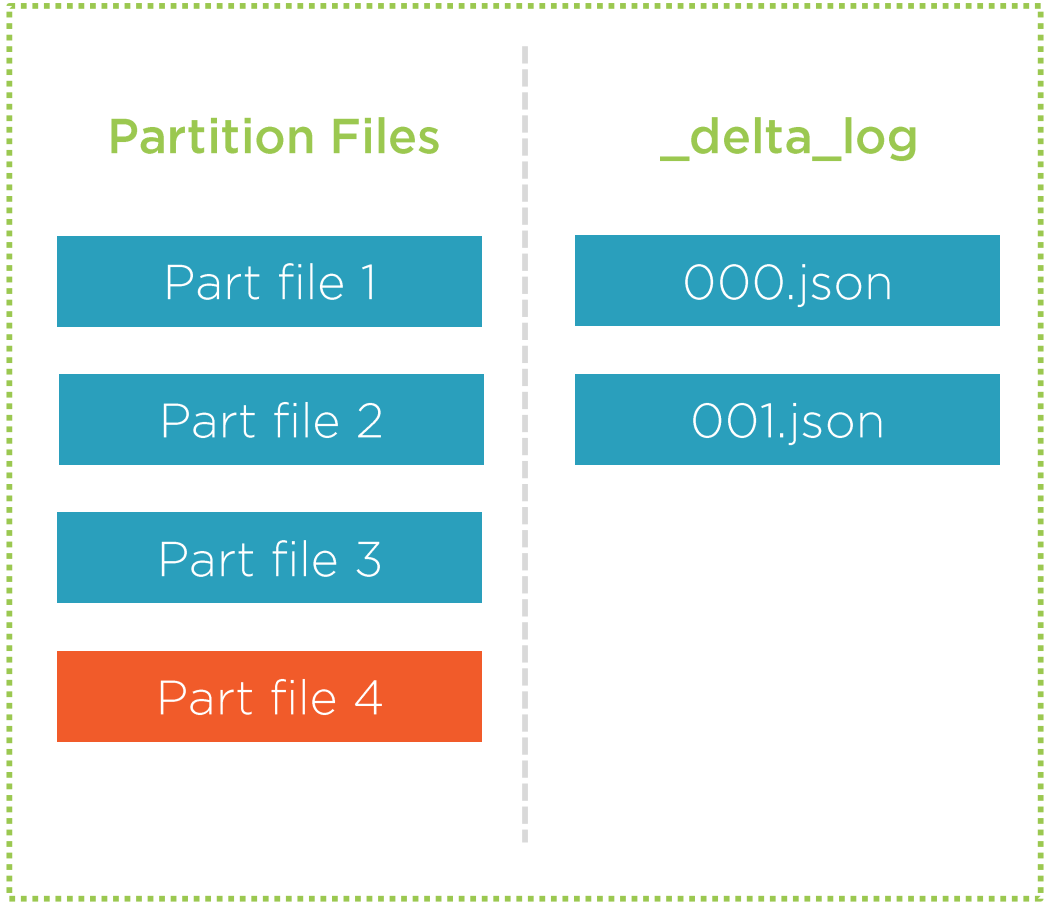
Read only Part files 1 & 2

Failure in Writing Data

Writer Process

1. Write partition files

Failure in writing new data via Part file 4



Delta Table on Data Lake

Reader Process

1. Read transaction log
2. Read partition files

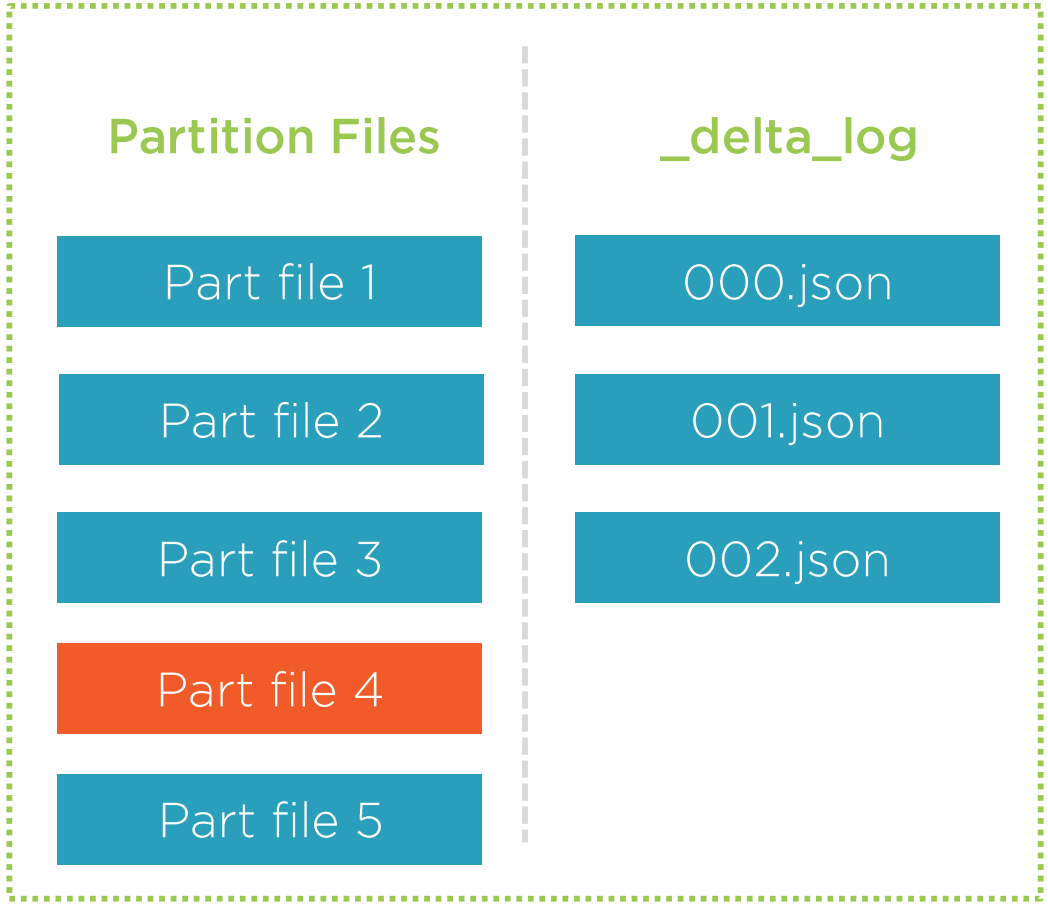
Read only Part files 1, 2 & 3

Updates

Writer Process

1. Write updated record (in file 1) in new partition file
2. Update log

Part file 5 is written with updated record, and added to log that part file 1 is not needed anymore



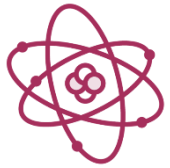
Delta Table on Data Lake

Reader Process

1. Read transaction log
2. Read partition files

Read only Part files 2, 3 & 5

Delta Lake Features



Delta lake allows to perform ACID transactions, using Serializable isolation levels

Underlying file format is Parquet and transaction log is stored in _delta_log folder

Data is only considered written to the table when transaction log is updated; also providing the full audit trail of changes

Helps in doing Merge, Update and Delete operations on the underlying files

Additional Features

Enforce Schema

Prevent bad data from being added to table

Time Travel

Access & revert to older versions of data

Z-Ordering

Sort data in files based on multiple columns

Statistics

Auto collect min & max values of columns

Garbage Collection

Clean up stale part files using VACUUM

Delta Lake Merge

```
%sql
MERGE INTO TaxiServiceWarehouse.DimTaxiZones AS tgt
USING StagingTaxiZones AS src
      ON tgt.LocationId = src.LocationId
WHEN MATCHED THEN
      UPDATE SET tgt.IsActive = src.IsActive
WHEN NOT MATCHED THEN
      INSERT (LocationId, IsActive...) VALUES (LocationId, IsActive...)
```

WARNING!!

Reading underlying delta file directly may cause issues, as it stores multiple versions of records

Use Cases

- Build data warehouse in data lake
- Slowly changing dimensions (SCDs)
- Change data capture
- Restore data in case of failures
- Common storage for batch & streaming data

Summary

Databricks APIs

- Workspace, Clusters, Pools, Jobs, DBFS, Libraries etc.

Invoke API using Powershell

- URI, Bearer Token, Http Method

Delta Lake

- Save DataFrame using delta format

Delta Lake scenarios

- Simultaneous read/write, Failure in writing, Updating records

Delta Lake features and use cases